



Universidade de Brasília
Faculdade UnB Gama
Disciplina: Estrutura de Dados 1 – EDA 1

Alocação Dinâmica de Memória

Aula sobre Ponteiros em C/C++

Prof. Filipe Tôrres

16 de abril de 2026

Introdução a Ponteiros

A variável na memória

O que é um Ponteiro?

Declaração de Ponteiros

Operações

Componentes de um Ponteiro

Parte de Endereçamento

Parte de Valor

Exemplos de Códigos

Exemplo 1: Endereço e Valor de uma Variável

Exemplo 2: Troca de Valores

Considerações Finais

Boas Práticas e Cuidados

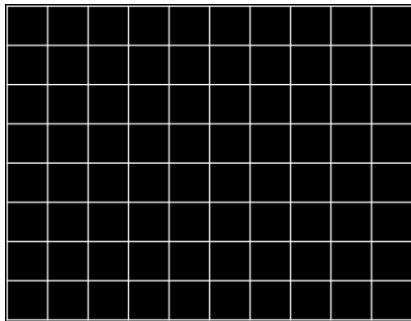
Conclusões

Introdução a Ponteiros

A variável na memória



```
#include <stdio.h>
int main() {
    return 0;
}
```



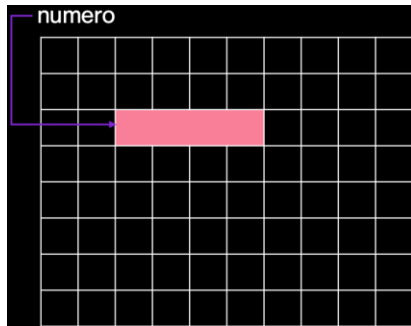
Área de memória do programa.

Introdução a Ponteiros

A variável na memória



```
#include <stdio.h>
int main() {
    int numero;
    return 0;
}
```



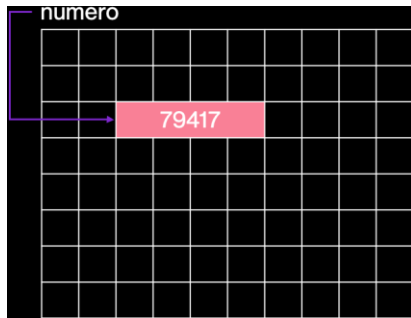
Área de memória do programa.

Introdução a Ponteiros

A variável na memória



```
#include <stdio.h>
int main() {
    int numero;
    numero = 79417;
    return 0;
}
```



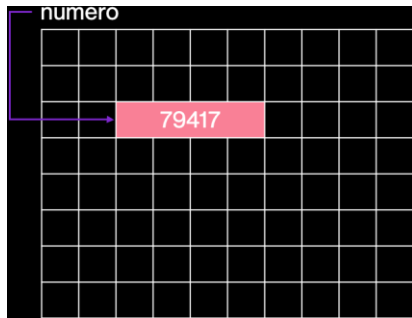
Área de memória do programa.

Introdução a Ponteiros

A variável na memória



```
#include <stdio.h>
int main() {
    int numero;
    numero = 79417;
    //Valor de 'numero'
    printf("Valor: %d\n", numero);
    //Endereço de 'numero'
    printf("Endereço: %p\n", &numero);
    return 0;
}
```



Área de memória do programa.

Introdução a Ponteiros

O que é um Ponteiro?



Um **ponteiro** é uma variável que armazena o endereço de memória de outra variável. Isso nos permite acessar e manipular diretamente a memória do sistema.

Em C/C++, os ponteiros são declarados usando a seguinte sintaxe:

```
1      tipo  *nome_do_ponteiro;
```

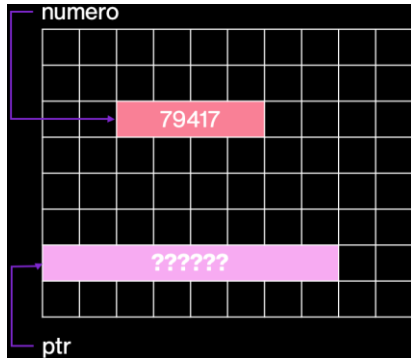
onde **tipo** é o tipo de dado ao qual o ponteiro aponta.

Introdução a Ponteiros

Operações: Declaração



```
#include <stdio.h>
int main() {
    int numero, *ptr;
    numero = 79417;
    return 0;
}
```



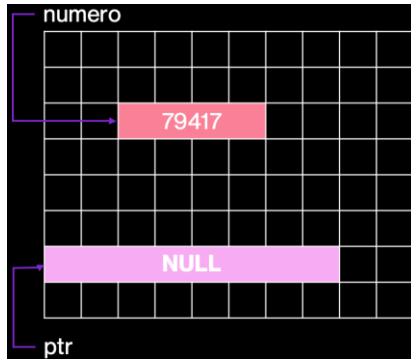
Área de memória do programa.

Introdução a Ponteiros

Operações: Inicialização



```
#include <stdio.h>
int main() {
    int numero, *ptr;
    numero = 79417;
    ptr = NULL;
    return 0;
}
```



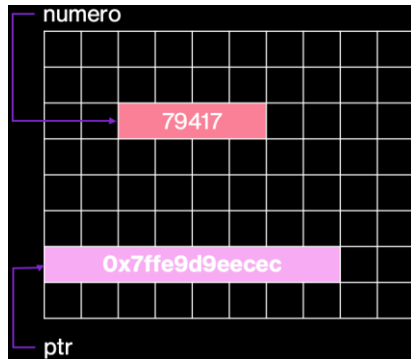
Área de memória do programa.

Introdução a Ponteiros

Operações: Atribuição



```
#include <stdio.h>
int main() {
    int numero, *ptr; numero = 79417;
    ptr = &numero; // Atribuindo o endereço de 'numero' para o ponteiro 'ptr'
    return 0;
}
```



Área de memória do programa.

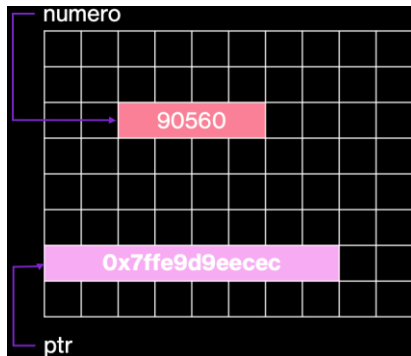
É mostrado a **atribuição de um endereço de memória** a um ponteiro. O **ponteiro ptr** agora armazena o **endereço de memória de numero**, permitindo que você acesse o valor de numero através de ptr. Isso é um conceito fundamental em C, permitindo **manipulação direta da memória** e **passagem de dados entre funções** de forma eficiente.

Introdução a Ponteiros

Operações: Atribuição



```
#include <stdio.h>
int main() {
    int numero, *ptr;
    numero = 79417;
    // Atribui o endereço de memória de 'numero' para o ponteiro 'ptr'
    ptr = &numero;
    // Atribui o valor 90560 ao endereço de memória apontado por 'ptr'
    *ptr = 90560
    return 0;
}
```



Área de memória do programa.

É mostrado uma das características mais poderosas dos **ponteiros**: a capacidade de **modificar diretamente o valor de uma variável** por meio do seu **endereço de memória**. Usando a operação de desreferência (`*ptr`), o ponteiro pode alterar o valor armazenado em `numero`, sem que seja necessário acessar diretamente a variável.

Essa abordagem é fundamental para a manipulação eficiente de dados em memória, especialmente quando se trabalha com **funções**, **estruturas de dados dinâmicas** e **gerenciamento de memória** em C.

Componentes de um Ponteiro

Parte de Endereçamento



Um ponteiro armazena o endereço de memória de uma variável. Isso é obtido usando o operador de endereço (&) e permite que trabalhem com o endereço real da variável.

Componentes de um Ponteiro

Parte de Valor



Para acessar o valor apontado por um ponteiro, usamos o operador de desreferência (*). Isso nos permite obter o valor armazenado na memória em um endereço específico.

Exemplos de Códigos

Exemplo: Endereço e Valor de uma Variável



```
1  #include <stdio.h>
2
3  int main() {
4      int numero = 79417;
5      int *ptr; // Declaração de um ponteiro para int
6
7      //Atribuição do endereço de 'numero' ao ponteiro ptr:
8      ptr = &numero;
9
10     //Impressão do endereço de 'numero'
11     // e seu valor usando o ponteiro
12     printf("Endereço de 'numero': %p\n", ptr);
13     printf("Valor de 'numero': %d\n", *ptr);
14
15     return 0;
16 }
```

Exemplos de Códigos

Exemplo: Troca de Valores



```
1  #include <stdio.h>
2  // Função para trocar o valor de duas variáveis float:
3  void trocarValores(float *ptr1, float *ptr2) {
4      float temp = *ptr1; //Armazena o valor de ptr1 em uma variáv
5      *ptr1 = *ptr2;      //Atribui o valor de ptr2 a ptr1
6      *ptr2 = temp;      //Atribui o valor temporário a ptr2
7  }
8  int main() {
9      float a = 3.14;
10     float b = 2.71;
11     printf("Valores originais: a = %.2f, b = %.2f\n", a, b);
12     //Chama a função para trocar os valores de 'a' e 'b':
13     trocarValores(&a, &b);
14     printf("Valores trocados: a = %.2f, b = %.2f\n", a, b);
15     return 0;
16 }
```


Exemplos de Códigos

Exemplo 1: Declaração de Ponteiros



```
int x = 10;  
int *p = &x; // ponteiro p armazena o endereço de x
```

```
// Imprime o valor de x  
printf("Valor de x: %d\n", x);
```

```
// Imprime o endereço de x  
printf("Endereço de x: %p\n", &x);
```

```
// Imprime o endereço de p (que é o de x)  
printf("Valor de p: %p\n", p);
```

```
// Imprime o valor de x através do ponteiro  
printf("Valor apontado por p: %d\n", *p);
```

- **Operador & (endereço):** Serve para obter o **endereço de memória** de uma variável.
- **Operador * (desreferenciação):** Usado para **acessar o valor armazenado no endereço** apontado por um ponteiro.

Exemplos de Códigos

Exemplo: Ponteiros e Arrays



```
int arr[] = {1, 2, 3, 4};
```

```
// Ponteiro aponta para o primeiro elemento de arr
```

```
int *p = arr;
```

```
printf("Primeiro elemento: %d\n", *p);    // Imprime 1
```

```
printf("Segundo elemento: %d\n", *(p + 1)); // Imprime 2
```

Exemplos de Códigos

Exemplo: Ponteiros e Funções



```
void incrementar(int *p) {  
    // Incrementa o valor apontado por p  
    (*p)++;  
}  
  
int main() {  
    int num = 10;  
    // Passa o endereço de num para a função  
    incrementar(&num);  
    // Imprime 11  
    printf("Valor de num após incremento: %d\n", num);  
  
    return 0;  
}
```

Exemplos de Códigos

Exemplo: Ponteiros e Alocação Dinâmica



```
// Aloca memória para um inteiro
int *p = (int*) malloc(sizeof(int));
if (p != NULL) {
    // Atribui 20 à memória alocada
    *p = 20;
    // Imprime 20
    printf("Valor alocado: %d\n", *p);
    // Libera a memória alocada
    free(p);
}
else {
    printf("Erro ao alocar memória\n");
}
```

- A função `malloc` (do cabeçalho `<stdlib.h>`) é utilizada para **alocar memória dinamicamente**. O que essa linha faz é alocar memória suficiente para armazenar **um único inteiro**.
- O operador `sizeof(int)` retorna o tamanho em bytes de um tipo `int`. Em sistemas modernos, geralmente o tamanho de um `int` é 4 bytes, mas pode variar dependendo da plataforma.
- `(int*)` faz um **cast** (conversão de tipo) para o tipo `int*`, que é um ponteiro para um inteiro.

Este código em C demonstra como **alocar memória dinamicamente**, **atribuindo valores** a essa memória e, em seguida, **liberando** a memória alocada quando não for mais necessária.

- ▶ Inicializar ponteiros antes de usá-los.
- ▶ Não tentar acessar áreas de memória não alocadas.

Lições Aprendidas

- ▶ O que são ponteiros.
- ▶ Como declarar um ponteiro.
- ▶ Fazer uso adequado da parte de valor do ponteiro
- ▶ Fazer uso adequado da parte de endereçamento do ponteiro
- ▶ Ter boas práticas de programação envolvendo ponteiros

Exercícios Práticos

- ▶ **Exercício 1:** Criar um programa que recebe o endereço de um número e imprime o valor desse número.
- ▶ **Exercício 2:** Implementar uma função que troca os valores de duas variáveis usando ponteiros.
- ▶ **Exercício 3:** Criar uma função que recebe um array e um ponteiro, e altera os valores no array.